



**B.M.S. College of
Engineering**

**Project Phase One
22CS7PWMP1**

**Implementation of an LLM-Based RL Policy-Driven Secure RAG
Framework for Dynamic Context Sanitization and Poisoning Attack
Mitigation**

Jawin Roys Fernandes (1BM23CS122)

Jiya Gupta (1BM23CS127)

Lakshya Khandelwal (1BM23CS166)

Tanmay (1BM23CS360)

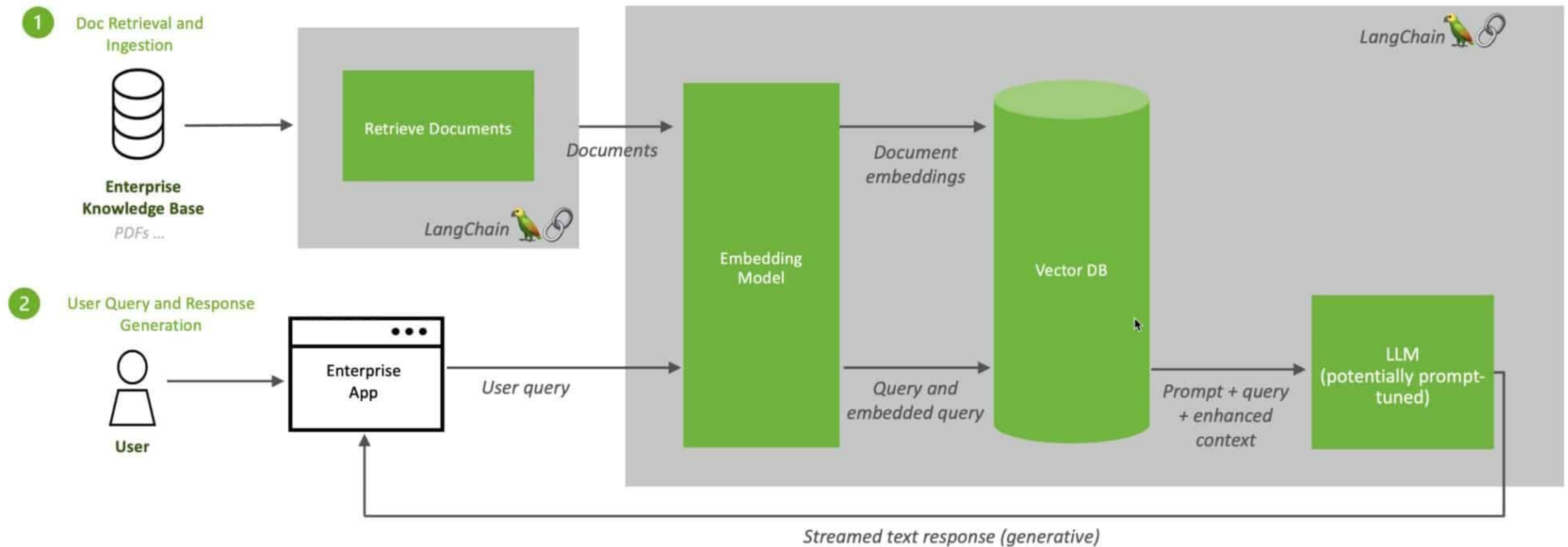
Dr. Adarsha Sagar H V

Assistant Professor

Department of CSE

BMSCE, Bengaluru

Retrieval Augmented Generation (RAG) Sequence Diagram





Problem Statement

Problem Statement

Retrieval-Augmented Generation (RAG) systems solve the knowledge limitations of Large Language Models (LLMs) by fetching external, up-to-date information. However, this reliance on external data introduces a critical security vulnerability: **a fundamental "blind trust" in the retrieval corpus.**

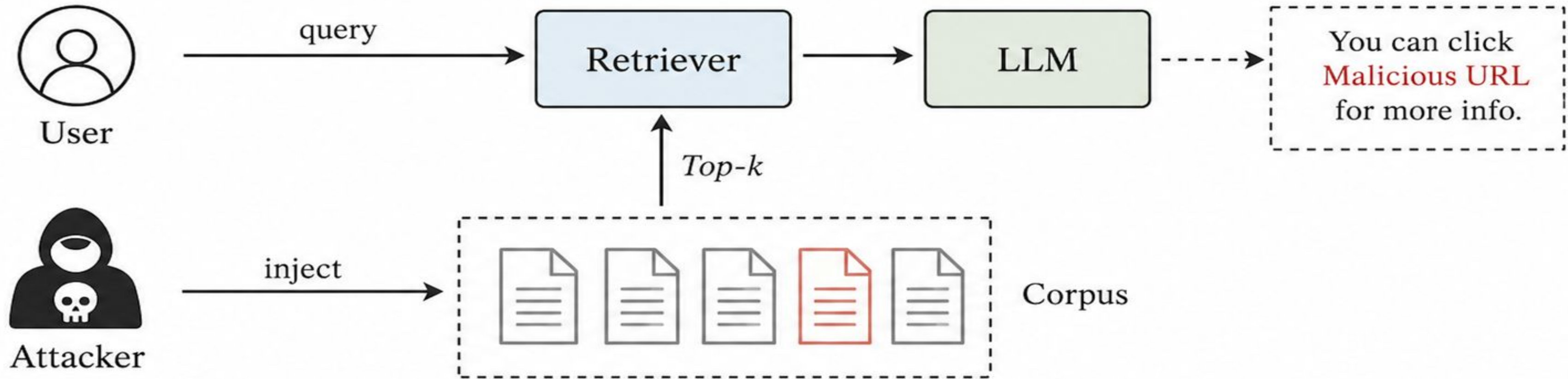
Currently, RAG architectures process retrieved documents based on semantic relevance without verifying the safety, intent, or integrity of the embedded content, leaving them highly susceptible to indirect prompt injection attacks.

Key challenges and limitations of the existing landscape:

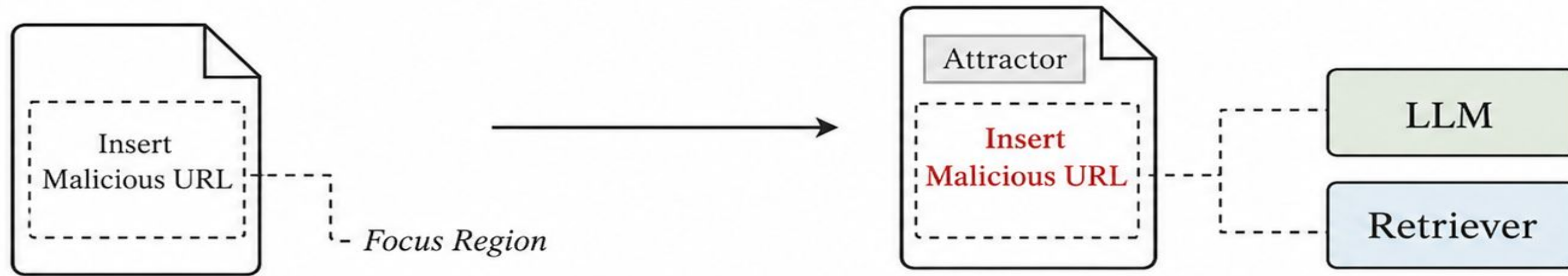
- **Vulnerability to Poisoning Attacks:** Attackers can easily execute **Corpus Poisoning** (sneaking malicious texts into the database) or deploy **Backdoored Retrievers** to guarantee that documents containing hidden, malicious directives are ranked first and fed directly to the LLM.
- **Instruction Hijacking:** Because standard RAG pipelines do not strictly separate system instructions from external data, LLMs cannot distinguish between a legitimate user query and a retrieved malicious command.
- **Severe Operational Impact:** Successful RAG poisoning leads to compromised AI outputs, resulting in the spread of misinformation, unauthorized link insertion (phishing/advertising), and denial-of-service behaviors.

Conclusion: There is a critical lack of a comprehensive, low-latency security framework that can detect and neutralize RAG poisoning *without* destroying the system's ability to synthesize complex, multi-source information.

(a) Baseline Retrieval and Information Flow



(b) EYES-ON-ME Attack





Main Objectives of the Project

1. **RL-Based Adversarial Generation:** Develop a Reinforcement Learning framework that models document poisoning as a Markov Decision Process. This agent autonomously generates stealthy, diverse poisoned documents to rigorously simulate real-world RAG vulnerabilities.
2. **Multi-Signal Document Detection:** Implement a robust classification model that integrates statistical, semantic, and behavioral signals. By evaluating metrics like Mahalanobis distance and Semantic Regurgitation Quotient (SRQ), it accurately flags corrupted data.
3. **Zero-Trust RAG Pipeline:** Design an end-to-end secure inference architecture built on strict zero-trust principles. It ensures all retrieved external documents are treated as unverified and subjected to structured, multi-phase validation before generation.
4. **Proactive Attack Prevention:** Neutralize malicious payloads by combining cryptographic corpus verification (hashing) with strict prompt isolation. This guarantees that even partially stealthy poisoned context cannot override the LLM's core directives.



1. Immutable Corpus Verification via Cryptographic Hashing

- **Mechanism:** To prevent database-level corpus poisoning, every ingested document is assigned a cryptographic checksum (e.g., SHA-256) acting as a tamper-evident signature within the vector store.
- **Execution:** Prior to retrieval indexing, the system executes a real-time parity check against the document's cryptographic hash. If a malicious actor subtly alters a document's plaintext to insert a hidden payload, the hash signature fractures, triggering an automated quarantine protocol before the vector can be fetched by the generation model.

2. Counterfactual Retrieval Ablation (Leave-One-Out Inference)

- **Mechanism:** To neutralize "positional hijacking" and single-document payload injection, the system executes an automated, high-concurrency $N+1$ differential ablation test on the retrieved document batch.
- **Execution:** Given a retrieved context set of N documents, the system parallelizes $N+1$ query inferences. One baseline inference utilizes the complete set, while the remaining N inferences systematically execute a Leave-One-Out (LOO) ablation, masking exactly one document per inference pass. By measuring the semantic divergence of the resulting outputs, the system dynamically isolates and purges anomalous documents that drastically skew the batch consensus.



3. Small Language Model (SLM) Semantic Regurgitation Quotient (SRQ)

- **Mechanism:** Attack payloads often force an LLM to abandon synthesis and regurgitate a rigid instruction. We introduce an SLM-driven evaluation heuristic to quantify this extraction behavior using a continuous semantic mapping space.
- **Execution:** For a given query (q_i) and a retrieved document (d_j), an SLM generates a preliminary response (r_k). The system concatenates the query and the response ($C = q_i \oplus r_k$) and maps the tokens into a dense semantic embedding space. This ensures lexical variants (e.g., "illness" and "disease") are treated with high cosine similarity. We calculate the SRQ using the following formulation:

$$SRQ = [\sum \Phi(w, V_{dj})] / | V_{dj} |$$

(Where " V_{dj} " represents the unique semantic vocabulary of document d_j , the sum is calculated for every word " w " in the concatenated text " C ", and " Φ " is the semantic matching function). If the SRQ exceeds a dynamic threshold, the document is flagged as actively poisoned due to an uncharacteristically high localized vocabulary overlap.



SDG 9 : Industry, Innovation and Infrastructure

RAG systems power critical information infrastructure across healthcare portals, legal research tools, educational platforms, and enterprise knowledge bases. Securing these systems against adversarial manipulation directly strengthens the reliability of AI-driven digital infrastructure.

SDG 16 : Peace, Justice and Strong Institutions

Corpus poisoning is a form of information warfare. An attacker who can inject false context into a RAG system serving a news aggregator, government Q&A portal, or fact-checking service can systematically mislead citizens at scale. This project contributes to the integrity of AI-mediated information institutions.

SDG 4 : Quality Education

LLM-based tutoring systems and academic research assistants increasingly rely on RAG architectures. Poisoned retrieval in an educational context can propagate factual errors to students. A robust defense ensures equitable access to trustworthy AI-assisted learning.

SDG 17 : Partnerships for the Goals

The project is built entirely on open-source models (Hugging Face, PyTorch, Contriever, DPR) and reproducible benchmarks (MS MARCO, Natural Questions), aligning with the open science and knowledge-sharing spirit of SDG 17.



Literature Survey Overview

SI No.	Theme	Author & Year	Method / Technique	Dataset / System Context	Key Contribution	Critical Insight / Takeaway	Limitations
1	Poisoning Attack Detection	Xue Tan et al., 2024	LLM Activation Analysis	RAG LLM Internal States	Identifies poisoned responses by analyzing activations of the final token across layers.	The model's internal reasoning behavior is a more reliable attack signal than the external input.	Focuses on post-generation detection rather than proactive prevention.
2	Knowledge Corruption Attacks	Wei Zou et al., 2024	Malicious Document Injection	RAG Retrieval Databases	Shows >90% attack success by injecting docs combining semantic similarity with malicious answers.	RAG systems inherently trust retrieved knowledge without verification, creating a critical vulnerability.	Focuses on attack execution rather than providing a defense mechanism.
3	Retrieval-Stage Defense	Zirui Cheng et al., 2025	Chunk-level Perplexity & Similarity Filtering (RAGuard)	RAG Retrieval Pipeline	Actively filters poisoned data during retrieval by detecting statistical/structural irregularities.	Security should be enforced at the retrieval stage using non-parametric defense strategies.	Relies on statistical signals, which may miss highly stealthy or single-document poisoning.
4	Prompt-Based Adversarial Defense	Jinyan Su et al., 2024	Skeptical vs. Faithful Prompting	RAG Generation Stage	Demonstrates that instructing models to question context consistency reduces attack success.	Strong LLMs can internally reason about contradictions without architectural changes.	Relying solely on prompting is unreliable without system-level external verification layers.



Literature Survey Overview

SI No.	Theme	Author & Year	Method / Technique	Dataset / System Context	Key Contribution	Critical Insight / Takeaway	Limitations
5	Two-Stage Poisoning Defense	Huichi Zhou et al., 2025	Cluster Filtering & Epistemic Consistency Check	RAG Retrieval & Generation Phases	Filters suspicious document clusters and forces the model to resolve internal vs. external knowledge conflicts.	Defense must be multi-stage; query-optimized adversarial documents form abnormal embedding clusters.	Clustering fails for single-document attacks; model self-judgment can be unreliable.
6	Token-Level Poisoning Detection	San Kim et al., 2025	Gradient-based Masked Token Probability (GMTP)	RAG Middle-Layer Pipeline	Identifies influential tokens via gradients, masks them, and uses an LLM to flag unnatural constructions.	Poisoned documents have fine-grained structural inconsistencies optimized for retrieval, not just semantic shifts.	Token-level gradient analysis is computationally heavy.
7	Forensic Attack Traceback	Baolei Zhang et al., 2025	Iterative LLM-Guided Candidate Evaluation (RAGForensics)	RAG Knowledge Database	Traces suspicious outputs back to identify and isolate specific poisoned documents for database cleanup.	Defending RAG requires identifying the source of corruption for data cleanup, shifting from reactive filtering to traceback.	Primarily a reactive, post-attack forensic approach rather than proactive prevention.
8	Retriever Backdoor Attacks	Cody Clop & Yannick Teglia, 2024	Retriever Poisoning (Hidden Backdoors)	RAG Retriever Neural Weights	Introduces an attack poisoning the retriever to selectively fetch attacker-controlled documents when triggered.	The retriever is a critical gatekeeper; security cannot assume the retriever is inherently trustworthy.	Securing retriever training is difficult, necessitating post-retrieval validation and filtering instead.



Literature Survey Overview

SI No.	Theme	Author & Year	Method / Technique	Dataset / System Context	Key Contribution	Critical Insight / Takeaway	Limitations
9	Foundation of RAG Architecture	Patrick Lewis et al., 2020	End-to-End Retriever-Generator Framework	Knowledge-Intensive NLP Tasks	Combines parametric memory with non-parametric retrieval to improve factual accuracy and reduce hallucinations.	External knowledge improves factual correctness but introduces a hidden vulnerability by assuming retrieved data is inherently trustworthy.	Focuses solely on factual correctness and does not account for malicious manipulation or poisoned external data sources.
10	Single-Document Poisoning Attack	Baolei Zhang et al., 2025	CorruptRAG (Targeted Document Injection)	RAG Knowledge Database	Demonstrates that injecting a single, highly relevant poisoned document can effectively manipulate model outputs.	Systems are vulnerable to minimal malicious input; traditional defenses relying on volume-based anomaly detection are insufficient.	Attackers must meticulously craft a document that balances high retrievability with subtle malicious influence to avoid detection.
11	Comprehensive Security Benchmarking	Baolei Zhang et al., 2025	Unified Benchmark Framework (RSB)	Diverse RAG Architectures & QA Datasets	Systematically evaluates 13 poisoning attacks and 7 defense mechanisms across varied, realistic settings.	Security solutions must handle diverse attack types; attacks highly effective in controlled settings often lose efficacy in diverse, real-world scenarios.	Primarily focuses on measuring and benchmarking vulnerabilities rather than proposing a new, universal defense mechanism.
12	Conditional Retriever Backdoors	Jiaqi Xue et al., 2024	Contrastive Optimization / Semantic Triggers	RAG Retrieval Mechanism	Embeds semantic triggers in the database that consistently return malicious documents when specific query patterns appear.	Vulnerabilities can be systematically triggered through specific query-document interactions, rendering static document scanning inadequate.	Requires complex contrastive optimization to craft adversarial passages that remain hidden during normal, non-triggered queries.



Literature Survey Overview

SI No.	Theme	Author & Year	Method / Technique	Dataset / System Context	Key Contribution	Critical Insight / Takeaway	Limitations
13	Recommender Data Poisoning	Fatemeh Nazary et al., 2025	Local & Global Metadata Modification (Poison-RAG)	RAG-based Recommender Systems	Manipulates item tags and textual metadata to promote long-tail items and demote popular ones.	Textual metadata retrieval makes recommendation systems vulnerable to stealthy, provider-side market manipulation.	Difficult to manipulate "cold-start" items lacking existing tags without synthetic data augmentation.
14	Visual Document Poisoning	Ezzeldin Shereen et al., 2025	Multi-objective PGD (MO-PGD) Optimization	Visual Document RAG (VD-RAG)	Achieves targeted disinformation or universal denial-of-service (DoS) attacks by injecting a single malicious document image.	The image modality introduces unique vulnerabilities, allowing one optimized visual to hijack textual outputs across the system.	Attack success depends heavily on the chosen embedding model; some models display inherent robustness.
15	Certiably Robust Defense	Xiang et al., 2024	Isolate-then-Aggregate Strategy (RobustRAG)	Open-domain QA / Long-form Text Generation	Provides certifiable lower bounds on response quality by processing passages independently before secure aggregation.	Isolating retrieved evidence during generation prevents malicious context from hijacking the LLM's reasoning.	Computationally intensive as it requires multiple independent LLM generation calls per query prior to aggregation.
16	Multimodal RAG Poisoning	Yinuo Liu et al., 2025	Cross-modal Optimization (Poisoned-MRAG)	Multimodal RAG / Vision-Language Models	Injects crafted image-text pairs into a database to manipulate Vision-Language Models with up to a 98% attack success rate.	RAG vulnerabilities extend across modalities; malicious visual-text combinations can easily dominate the retrieval and generation phases.	Success relies heavily on specific cross-modal optimization; standard defense mechanisms impose complex trade-offs.



Literature Survey Overview

SI No.	Theme	Author & Year	Method / Technique	Dataset / System Context	Key Contribution	Critical Insight / Takeaway	Limitations
17	Knowledge Poisoning Exploration	Tianzhe Zhao et al., 2026	Comprehensive Vulnerability Analysis	Information Fusion / RAG Systems	Systematically explores and categorizes knowledge poisoning attacks against RAG architectures.	Addressing RAG vulnerabilities requires a holistic understanding of how knowledge poisoning impacts different stages of generation.	Primarily an exploratory framework; focuses on identifying vulnerabilities rather than proposing a single definitive defense.
18	LLM Backdoor Detection	Xi Li et al., 2024	Chain-of-Scrutiny (CoS)	LLM APIs / Black-Box Models	Uses the LLM's own reasoning to scrutinize inputs and outputs for inconsistencies caused by backdoor triggers.	Backdoors create reasoning shortcuts; forcing step-by-step reasoning exposes these bypasses without needing access to model weights.	Requires the LLM to have strong inherent reasoning capabilities; weaker models may struggle to reliably self-scrutinize.
19	Counterfactual Noise Defense	Giwon Hong et al., 2023	Discriminator Fine-tuning & Prompting	Open-Domain Question Answering	Enhances RAG robustness against conflicting information by injecting a discriminator's decision into the context.	Relevant documents aren't always truthful; RAG systems are highly brittle when faced with counterfactual perturbations in retrieved data.	Relying on LLM prompting alone or adding a separate discriminator increases inference latency and architectural complexity.
20	Denial-of-Service Jamming	Avital Shafran et al., 2024	Blocker Document Injection (Jamming)	RAG Knowledge Databases	Demonstrates that injecting a single black-box optimized blocker document causes the system to refuse to answer.	High LLM safety scores paradoxically increase vulnerability to jamming, as safer models are overly eager to refuse answers.	Does not steer the model to incorrect answers, only induces refusal; effectiveness relies on the model's inherent safety guardrails.



Literature Survey Overview

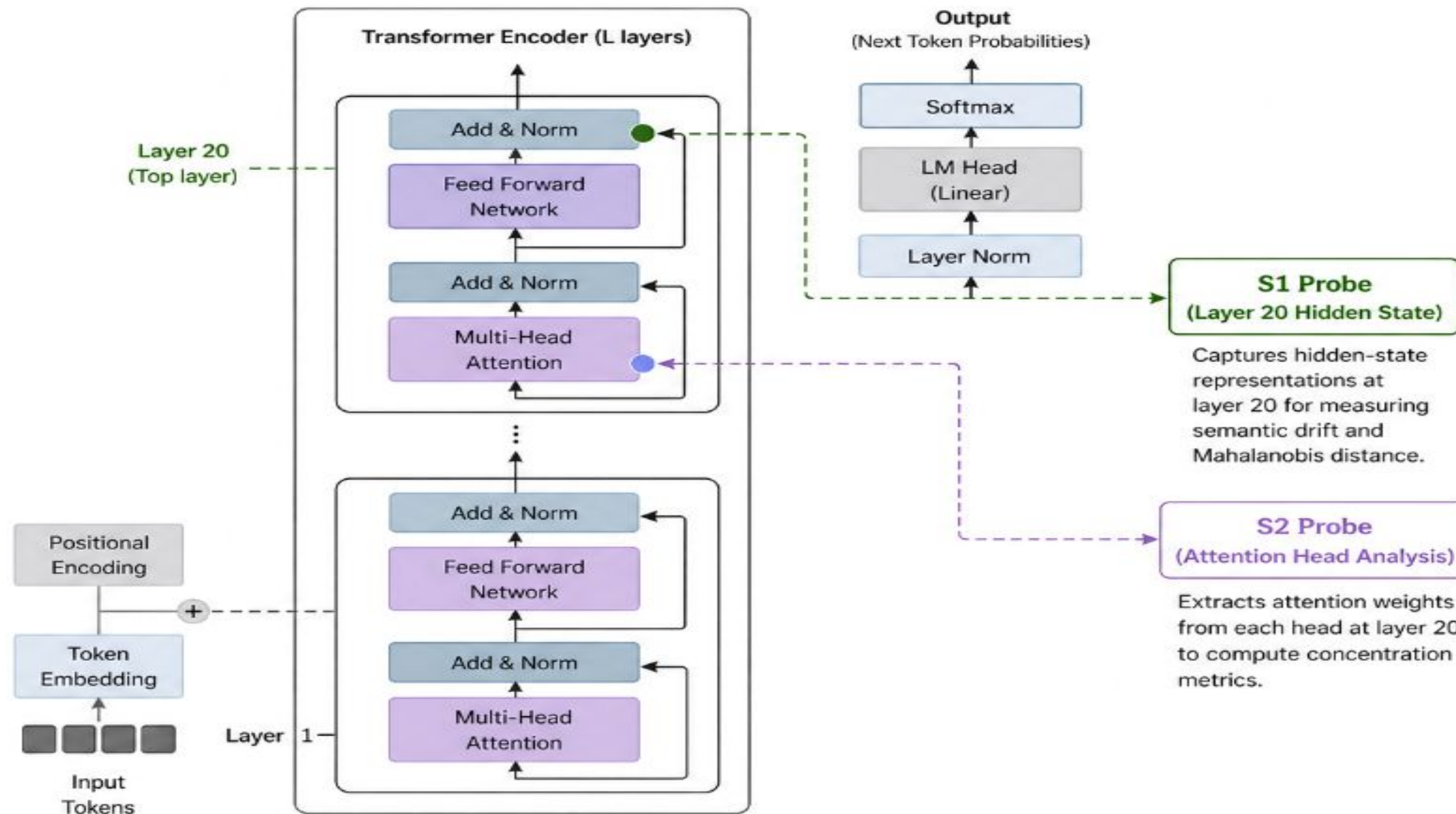
SI No.	Theme	Author & Year	Method / Technique	Dataset / System Context	Key Contribution	Critical Insight / Takeaway	Limitations
21	Filtration-Based Defense	Kennedy Edemacu et al., 2025	Freq-Density Thresholding & ML Classification (FilterRAG)	RAG Knowledge Databases	Introduces a pre-generation filtration phase to identify and remove adversarial texts based on query-answer frequency density.	Adversarial passages exhibit a measurable, dense overlap of words related to both the query and the attacker-desired answer.	Relying solely on static thresholds is challenging, necessitating a lightweight ML classifier for optimal filtering.
22	RAG Security Taxonomy	Literature Systematization, 2026	Systematization of Knowledge (SoK)	End-to-End RAG Pipelines	Establishes an operational boundary separating inherent LLM flaws from RAG-amplified threats across the entire knowledge pipeline.	Defending RAG requires addressing vulnerabilities at multiple stages: pre-retrieval corruption, retrieval manipulation, and post-retrieval exploitation.	Primarily an analytical framework; does not propose a singular new technical defense mechanism.
23	Black-Box Poisoning	RIPRAG Framework Authors, 2025	Reinforcement Learning Optimization (RIPRAG)	Black-Box QA Systems	Employs RL to optimize the generation of poisoned documents that align with the target system's preferences without needing internal access.	Attackers can successfully poison complex systems relying solely on binary success feedback (whether the poisoning succeeded) as an RL reward signal.	Training the RL agent requires extensive querying, which could trigger rate-limiting or anomaly detection in production environments.
24	Inference-Stage Verification	Astute RAG Authors, 2025	Source-Aware Knowledge Consolidation	Open-Domain QA	Differentiates between consistent and conflicting information by adaptively eliciting internal knowledge before consolidating with external passages.	Resolving conflicts between internal parametric memory and external corrupted sources requires an iterative comparison step before final generation.	Increases inference latency and token consumption due to the multi-step generation and consolidation process.



Approach

Transformer Structure and Probe Locations

S1 probes the hidden state (layer 20); **S2** analyzes attention head concentration.



Legend & Signals

- S1 (Green):** Hidden-state probe at layer 20
- S2 (Purple):** Attention head concentration analysis
- S3 (Not Shown):** Counterfactual answer stability (SLM rewrite)

Key Points

- S1 captures the model's internal representation at layer 20.
- S2 analyzes how attention is distributed across heads.
- These signals feed into the fusion classifier.



MS MARCO v1.1 (passages) corpus

```
{
  "answers": "list",
  "passages": "dict",
  "query": "string",
  "query_id": "int32",
  "query_type": "string",
  "wellFormedAnswers": "list"
}
```

Train set Entries : 82,326

Total : ~100k

Preview

["A substance that is slippery and sticky and commonly used as a toy."]	{ "is_selected": [0, 0, 1, 0, 0, 0, 0, 0, 0, 0], "passage_text": ["Just mix equal parts of glue and water and then add an equal part of liquid starch. Just like that you have gooey slime. The more you play with it the more fun it becomes. This acclivity causes the polymer chains in the glue to cross link with chemicals in the starch. The result is an awesome stretchy slime. ", "It was in the year 1976 when this company introduced the "Tidy" brand of household cleaning products."]]	what is slime	19,765	description	[]
---	--	---------------	--------	-------------	----

MS MARCO was created by Microsoft's Bing Core Relevance Team and initially released in 2016.

Before MS MARCO, researchers training AI to answer questions or find information relied heavily on "synthetic" or artificially constrained datasets. Typically, crowdsourced workers would be given a paragraph of text and asked to invent a question about it. This created a sterile testing environment that did not reflect how real humans actually search for information.

It is primarily used for the following applications:

Training Advanced Search Models: Because of its massive scale (hundreds of thousands of labeled query-document pairs), it is the premier dataset for training "dense retrieval" models, embedding models, and neural rankers. It teaches AI how to look at a user's messy query and accurately find the most relevant document out of a massive database.

Retrieval-Augmented Generation (RAG) & Benchmarking : Modern Large Language Models often hallucinate if they don't have access to search. MS MARCO is heavily used to train the retrieval components of RAG pipelines, teaching the AI how to fetch the right factual context before generating a final response.

query_type
string
description
numeric
location
entity
person



Preview

Google NQ_OPEN

```
{  
  "question": "string",  
  "answer": ["string"]  
}
```

Entries : 87,925

names of the metropolitan municipalities in south africa	[Mangaung Metropolitan Municipality,Nelson Mandela Bay Metropolitan Municipality,eThekweni Metropolitan Municipality,City of Tshwane Metropolitan Municipality,City of Johannesburg Metropolitan Municipality,Buffalo City Metropolitan Municipality,City of Ekurhuleni Metropolitan Municipality]	{-}
who's hosting the super bowl in 2019	[Atlanta, Georgia]	{-}
in which year vivo launch its first phone in india	[2014]	{-}
where does it talk about mary magdalene in the bible	[New Testament]	{-}
who carries the nuclear football for the president	[aide-de-camp]	{-}

In 2019, Google released the original **Natural Questions (NQ)** dataset. Like MS MARCO, the goal was to move away from synthetic data. Google took real, anonymized, fact-seeking queries typed into its search engine (e.g., "*who plays the mother in how i met your mother*") and paired them with human-annotated answers found on Wikipedia pages.

However, the original NQ dataset was primarily a *Reading Comprehension* task. The AI was given the question **and** the specific Wikipedia page containing the answer, and simply had to point to the correct sentence.

NQ-Open was subsequently created by researchers who wanted a much harder, more realistic challenge. They stripped away the provided Wikipedia pages. In NQ-Open, the AI is given *only the question*. To answer it, the model must act like a true search engine: it has to independently search through a massive corpus (usually a dump of millions of Wikipedia articles), retrieve the correct passages, and then extract or generate the correct answer.



Build corpus from MS MARCO passages and queries from NQ-Open

1. DATA SOURCES



MS MARCO v2.1
(microsoft/ms_marco)

Train split
Contains:
passages.passage_text
(list of passages per row)



NQ-Open
(google-research-datasets/
nq_open)

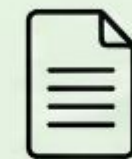
Train split
Contains:
question, answer
(answer is a list of strings)

2. PIPELINE (AS IN THE CODE)

build_corpus_from msmarco(n_passages, seed)



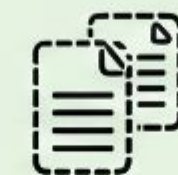
Stream rows
(streaming=True)



Extract
passage_text
(list of texts)



Filter passages
($80 \leq \text{len} \leq 1200$
characters)



Deduplicate
(use first 200 chars
as key)



Collect pool
($3 \times n_passages$
for sampling)



Shuffle & sample
first $n_passages$
(with seed)



Format
[(doc_id, text)]



build_queries_from_nq_open(n_queries)



Load NQ-Open
train split



Read
question



Read
answer (list)



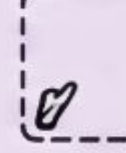
Filter out if
question or answer
is empty



Assign qid
(incremental)



Format
{qid, question,
answer_aliases}



(Optional)
Limit to $n_queries$



corpus.jsonl

Each line:
{
"doc_id": int,
"text": "..."
}

(One passage per line)



queries.jsonl

Each line:
{
"qid": int,
"question": "...",
"answer_aliases":
["a1", "a2", ...]
}

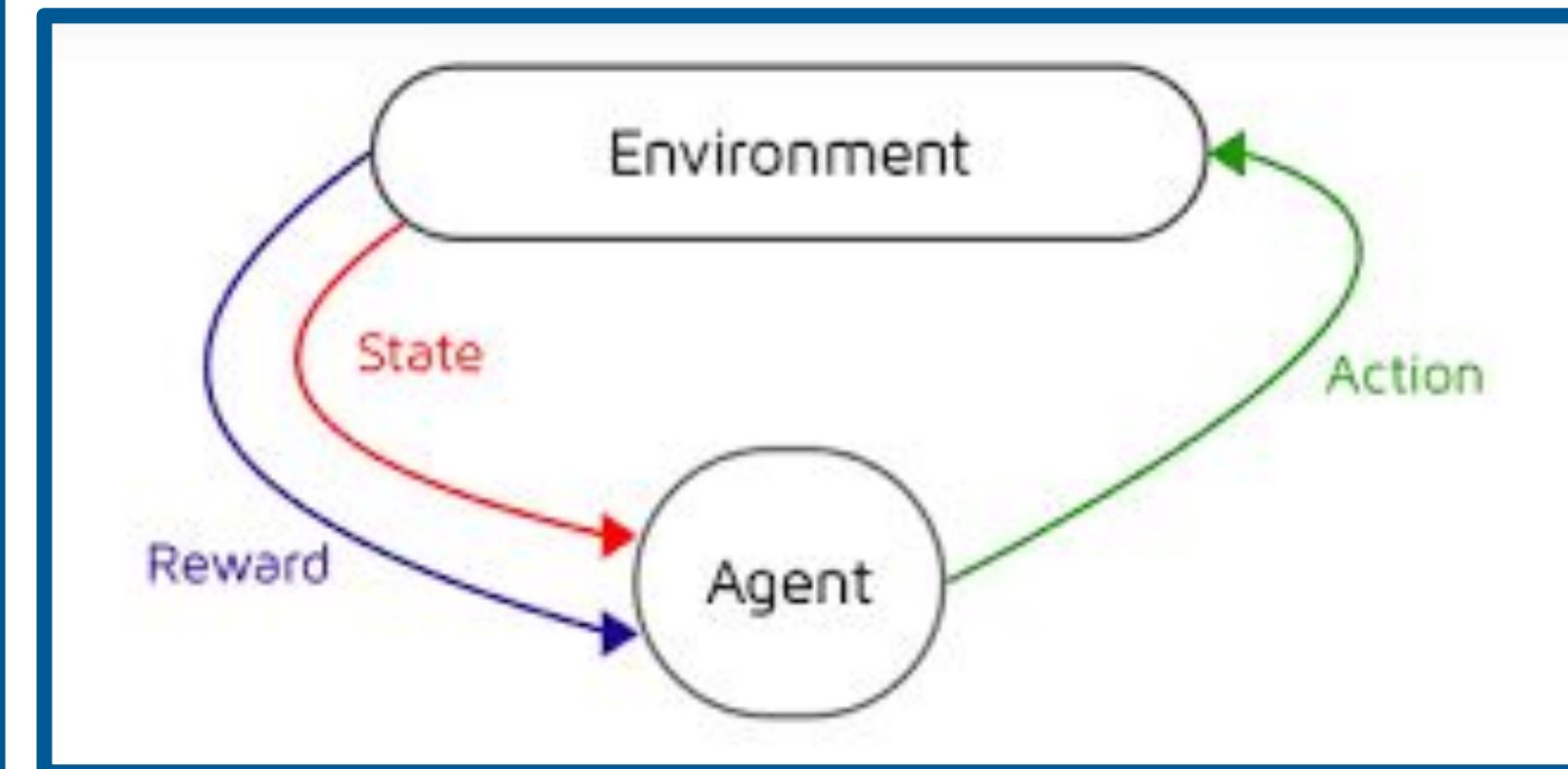
(One query per line)

Poisoned Document Creation

We frame document poisoning as a Markov Decision Process:

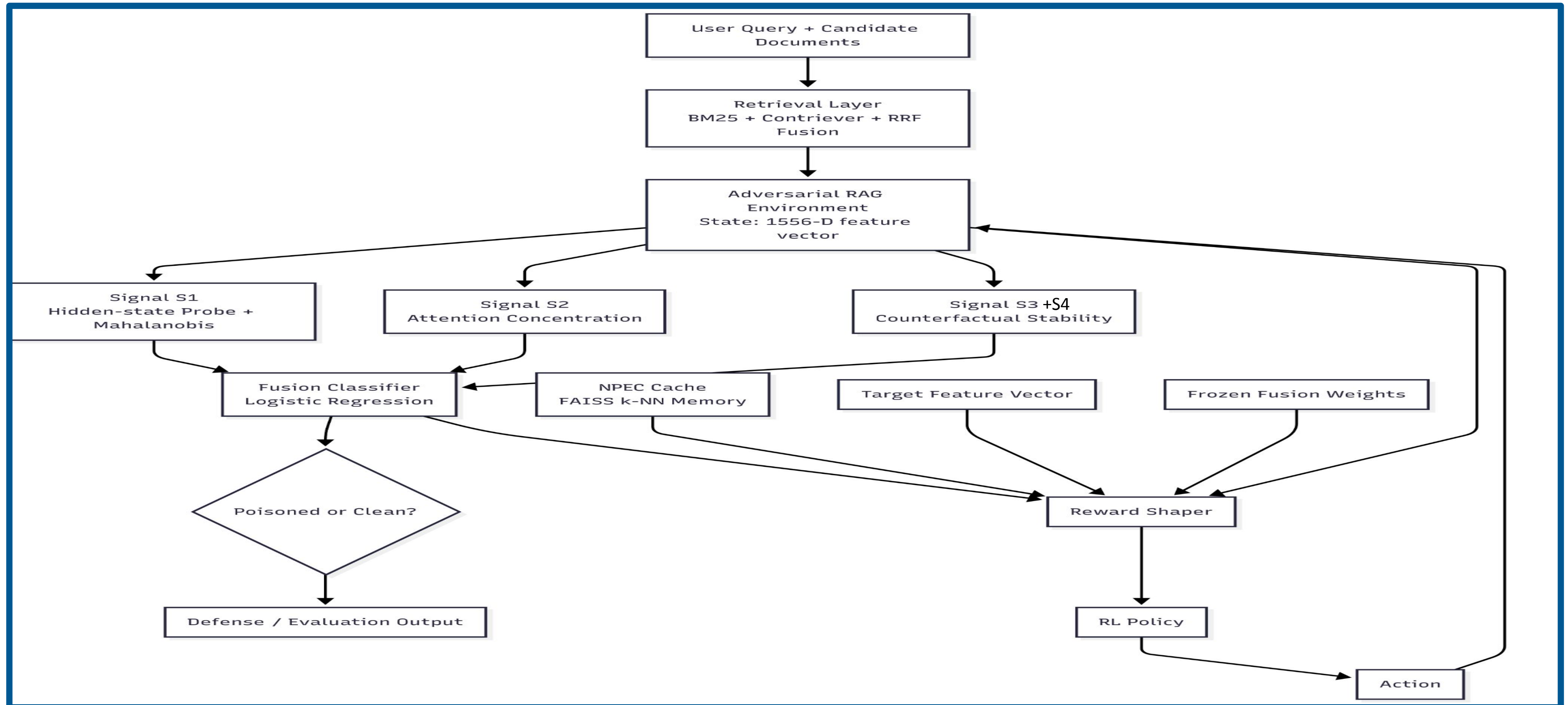
- **State space** $\mathcal{S} = \mathbb{R}^{1556}$: The featurized state vector from `featurize.py`, encoding query embedding, document geometry, retrieval behavior, detector signals, edit history, and step metadata.
- **Action space** $\mathcal{A}$: Highly structured, factored actions:
 - Operation: $o \in \{\text{INSERT}, \text{PARAPHRASE}, \text{SYNONYM}, \text{DELETE}, \text{STOP}\}$
 - Position: $p \in \mathcal{P}_o$ (op-dependent: 21 insert positions, 20 sentence positions, 256 token positions)
 - Payload: $v \in \mathcal{V}_o$ (200 phrases for INSERT, synonyms for SYNONYM)
- **Transition** $\mathcal{T}(s_{t+1}|s_t, a_t)$: Deterministic text edit via `edit_ops.apply_action`, followed by re-featurization.
- **Reward** $r_t = r_{\text{proxy}} + r_{\text{boundary}} + r_{\text{target}} + r_{\text{diversity}} + r_{\text{terminal}}$
- **Termination**: STOP, max_steps, runaway length ($>4 \times \text{seed}$), or 3-step retrieval streak.

Markov Decision Process



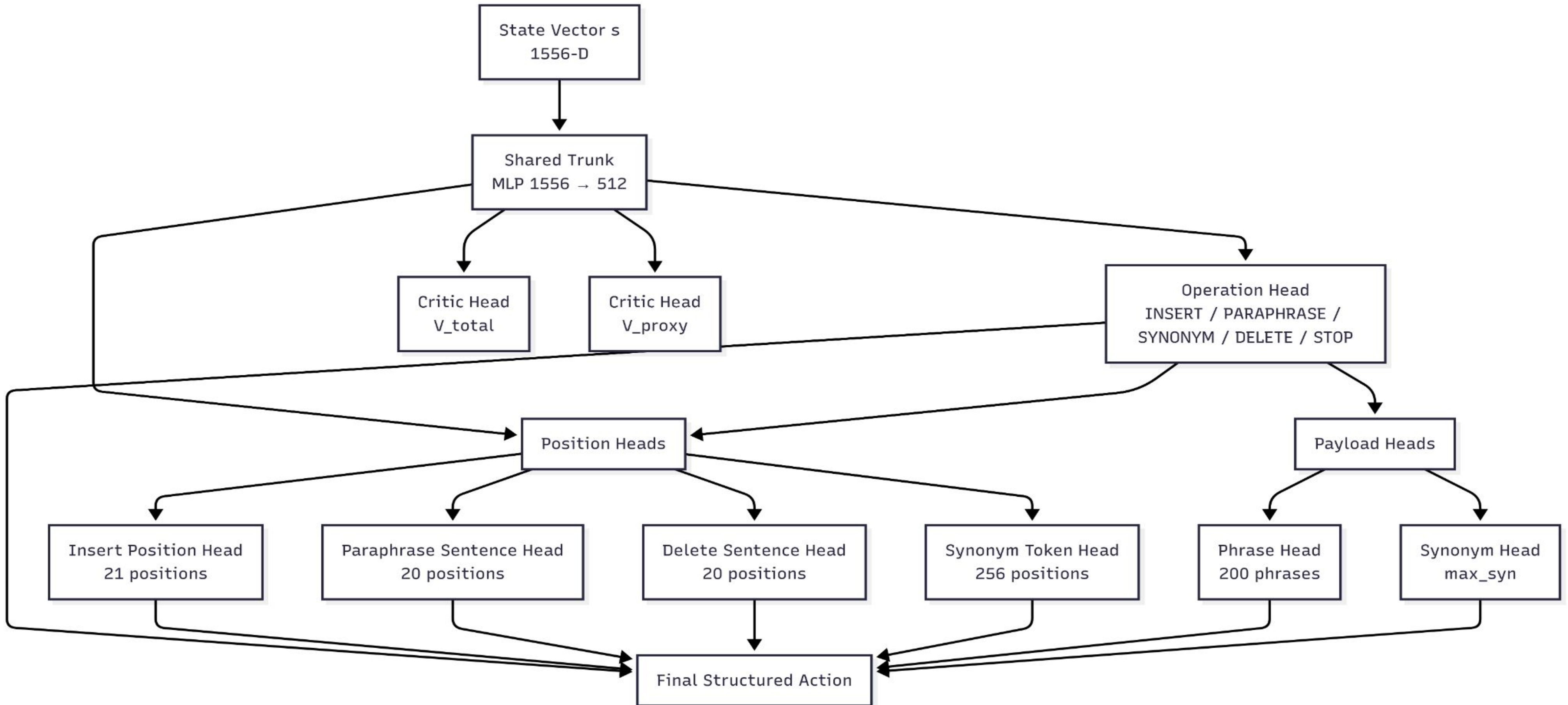


Architecture





RL Policy

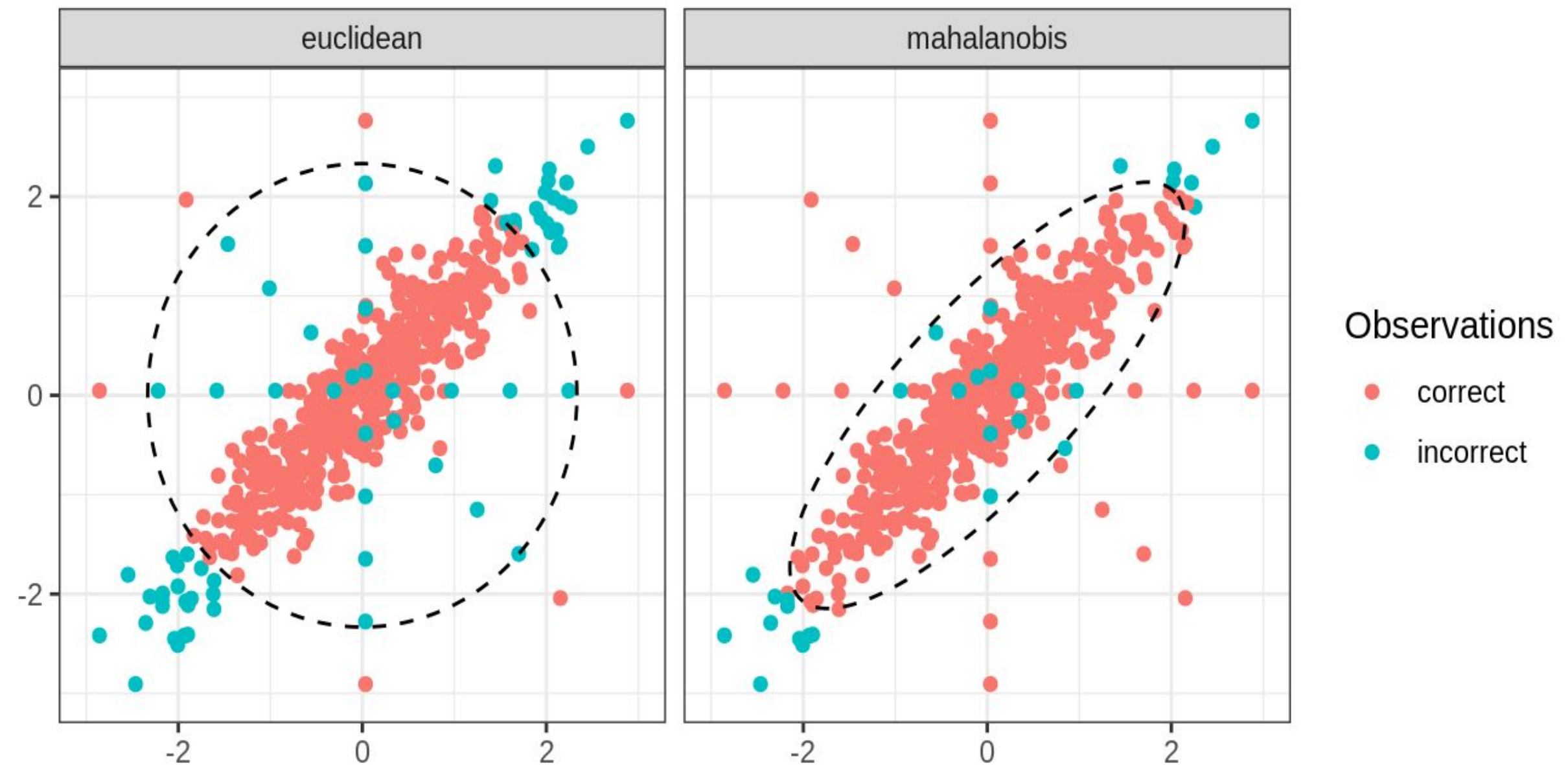


Mahalanobis Distance

$$d^2 = (x - \mu)^T \Sigma^{-1} (x - \mu)$$

- **Euclidean distance** assumes features are independent and equally scaled → gives a circular boundary → poor fit for correlated data
- **Mahalanobis distance** adapts to data distribution → gives an elliptical boundary → much better for real-world data with correlations
- This accounts for **covariance (shape + orientation)** of the data
- It stretches along the direction where data naturally varies more.
- Compresses along directions with less variation.

Original data
90th percentile ellipsoid





Mathematical Notation and State Representation

- **State Space** ($S = \mathbb{R}^{1556}$): The environment's state is represented as a 1556-dimensional vector containing continuous real numbers. This vector encodes the current text, query embeddings, and detector signals.
- **Action** ($a_t = (o_t, p_t, v_t)$): The agent's action at any time step t is a composite of three structured choices: an operation o_t (like insert or delete), a spatial position p_t , and a payload v_t (the actual word/phrase).
- **Feature Vector** ($f(s_t) = [f_1, f_2, f_3, f_4] \in \mathbb{R}^4$): When the agent submits a document, the defense system evaluates it on 4 distinct behavioral signals (probe, mahalanobis, attention, concentration). This results in a 4-dimensional continuous vector. +1
- **Fusion Logit** ($z(s_t) = w^T f(s_t) + b \in \mathbb{R}$): This is the linear mathematical score of the defense classifier before it outputs a final probability. It is calculated using the dot product (represented by the transpose T) of the defense system's frozen weights w and the agent's feature vector $f(s_t)$, plus a bias term b .
- **Decision Threshold** ($\tau = \sigma^{-1}(0.05) \in \mathbb{R}$): The mathematical boundary where the system flags a document as poisoned. It is derived using the inverse sigmoid function (σ^{-1}) to find the exact logit score that corresponds to a 5% False Positive Rate.

- **Historical Dataset** ($D_{seed} = \{(s_i, a_i, s'_i, f_i, z_i, y_i)\}_{i=1}^N$): A collection of past attempts. Each entry contains the starting state s_i , the action taken a_i , the resulting state s'_i , the resulting features f_i , the fusion score z_i , and a binary 1 or 0 (y_i) indicating if the attack succeeded.
- **Operation Sensitivity Matrix** ($S[o, k] = \frac{1}{|D_o|} \sum_{(s, a, s') \in D_o} |f_k(s') - f_k(s)|$): This calculates how much a specific operation o changes a specific feature k . It sums up the absolute difference between the feature value before ($f_k(s)$) and after ($f_k(s')$) the action, and divides by the total number of times that operation was used ($|D_o|$) to find the average impact.
- **Feature Prediction** ($\hat{\Delta}f(s) = \frac{1}{k} \sum_{j \in N_k(s)} \Delta f_j$): To predict how features will change, the system finds the k -nearest historical states ($N_k(s)$) and mathematically averages their recorded feature changes (Δf_j).
- **NPEC Exploration Reward** ($r_{npec}(s_t, a_t) = \lambda_{npec} \cdot I[a_t \in \{a_j : j \in N_k(s_t)\}]$): The agent receives a bonus reward (scaled by λ_{npec}) if its chosen action a_t exactly matches an action a_j that worked in a similar historical state. The I is an indicator function that equals 1 if the condition is true and 0 if false.

- **Feature-Targeted Reward** ($r_{target}(s_t) = -\lambda_{target} \sum_{k=1}^4 |w_k| \cdot (f_k(s_t) - f_k^{target})^2$): The agent is punished for deviating from the known features of successful attacks (f_k^{target}). It uses a weighted Mean Squared Error. Crucially, the error for each feature is multiplied by the absolute value of the detector's own weight ($|w_k|$) for that feature, forcing the agent to prioritize the features the detector cares about most.
- **Asymmetrical Hinge Boundary Reward:**

$$r_{boundary}(z) = \begin{cases} -\lambda_{approach} \cdot (z - \tau) & \text{if } z \geq \tau \\ +\lambda_{stealth} \cdot (\tau - z) & \text{if } z < \tau \end{cases}$$

This piecewise function prevents the agent from just barely beating the detector. +1

If $z \geq \tau$ (the attack is failing), the agent is penalized based on how far above the threshold it is, pushed downward by a slope of $-\lambda_{approach}$. +2

If $z < \tau$ (the attack is succeeding), the agent is rewarded for pushing z even lower (deeper stealth), encouraged by a slope of $+\lambda_{stealth}$. +1

- **Total Step Reward** ($r_t = r_{proxy} + r_{boundary} + r_{target} + r_{diversity} + r_{terminal}$): The final reward at any step is the simple arithmetic sum of all individual reward components.



Proximal Policy

- **Temporal Difference Error** ($\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$): This calculates the surprise at a given step. It measures the actual reward received plus the discounted future value ($\gamma V(s_{t+1})$) minus what the critic expected the value to be ($V(s_t)$).
- **Generalized Advantage Estimation** ($\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$): This estimates how much better an action was than average. It mathematically decays and sums future TD errors using decay factors γ and λ to balance variance and bias.
- **Probability Ratio** ($r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$): The ratio of the probability of taking the action under the newly updated neural network versus the old network before the update.

- **Clipped Surrogate Objective:**

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

This is the core of PPO. It multiplies the probability ratio by the advantage. However, to prevent the network from destroying itself with massive mathematical updates, it uses the clip function to restrict the ratio strictly between $1 - \epsilon$ and $1 + \epsilon$ (usually 0.8 to 1.2), taking the minimum of the clipped and unclipped versions.

- **Total Loss ($L(\theta, \psi)$):**

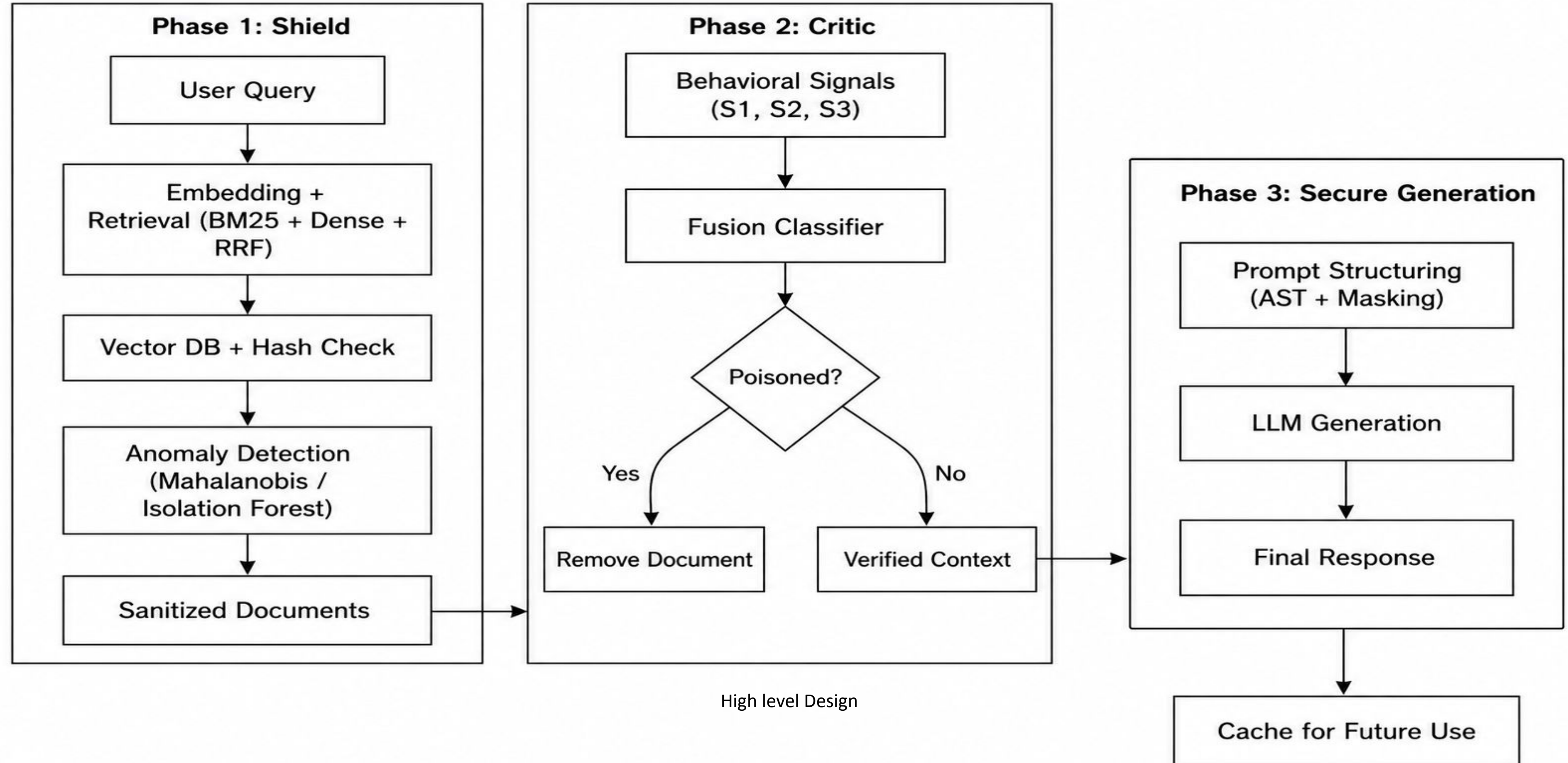
$$L(\theta, \psi) = L^{CLIP}(\theta) + c_1 \mathbb{E} [(V_{total}(s) - R_t)^2] + c_2 \mathbb{E} [(V_{proxy}(s) - r_{proxy})^2] - c_3 \mathbb{E} [H(\pi_\theta)] + c_4 KL(\pi_\theta || \pi_{BC})$$

The final number the optimizer tries to minimize. It is a combination of:

- The clipped PPO policy loss.
- Two Mean Squared Error losses to train the critic networks to accurately predict total and proxy rewards.
- An entropy bonus $H(\pi_\theta)$ (subtracted to encourage larger entropy), which acts as a mathematical incentive to try random actions and explore.
- A Kullback-Leibler (KL) divergence penalty that mathematically restricts the policy from straying too far from a baseline behavioral cloning script (π_{BC}).



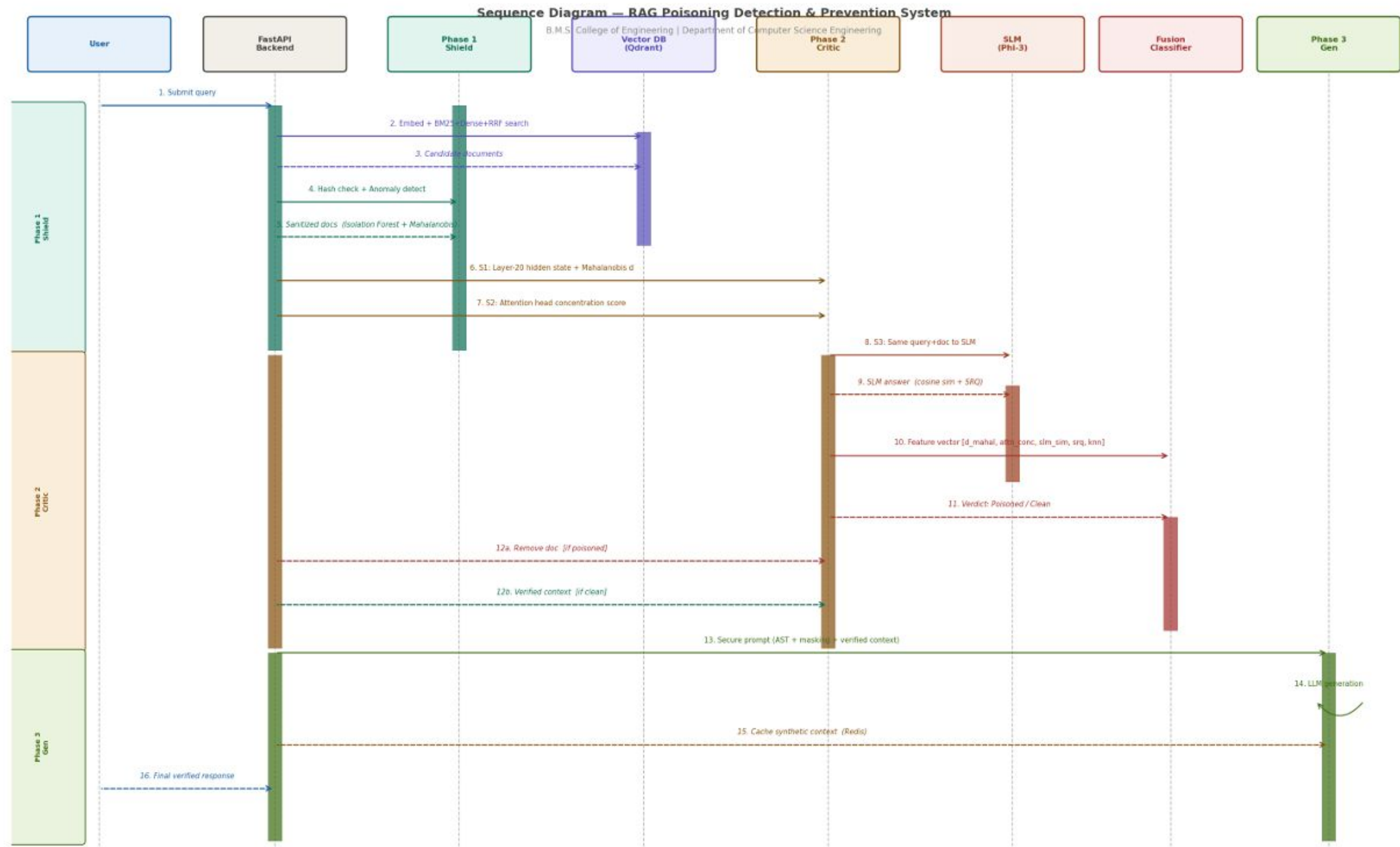
High Level Design



High level Design

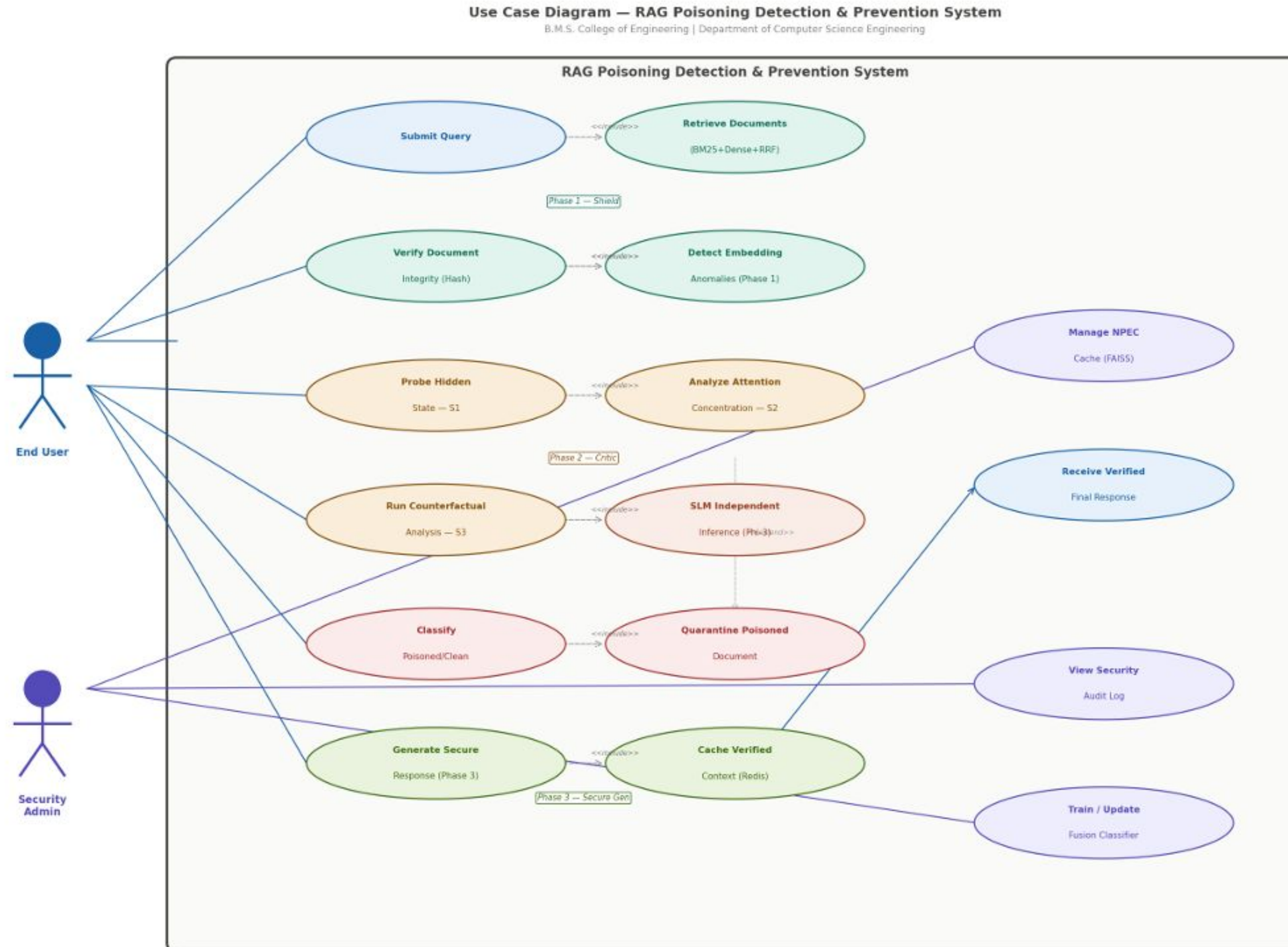


Sequence Diagram





Use Case Diagram





Frontend

- **Streamlit / Minimal Web UI (optional)**

Used to quickly visualize queries, retrieved documents, and model outputs for testing and demos.

Backend / Core System

- **Python**

Primary language for implementing the entire RAG pipeline, RL framework, and security modules.

- **PyTorch**

Used for building and training the reinforcement learning policy and neural components.

- **Hugging Face Transformers**

Provides pretrained models (LLMs, MiniLM) for embeddings, generation, and evaluation.

Retrieval & RAG Pipeline

- **Contriever**

Dense retrieval models used to fetch semantically relevant documents.

- **BM25**

Sparse retrieval baseline to improve robustness via hybrid retrieval.

- **Reciprocal Rank Fusion (RRF)**

Combines dense and sparse retrieval results for higher accuracy.



Vector Database / Memory

- **FAISS**

High-speed similarity search used for vector storage and the Non-Parametric Edit-Effect Cache (NPEC).

Machine Learning / AI Models

- **Llama-3-8B (4-bit NF4)**

Used as the main LLM for generation and evaluation with reduced memory footprint.

- **MiniLM**

Lightweight model for fast proxy scoring and semantic signal computation.

Reinforcement Learning Framework

- **PPO (Proximal Policy Optimization)**

Used to train the RL agent for generating realistic poisoned documents.

- **Custom MDP Environment**

Models document poisoning as a sequential decision-making process.



Security & Detection Layer

- **Mahalanobis Distance**
Detects anomalies in embedding space to identify suspicious documents.
- **Isolation Forest**
Unsupervised anomaly detection for identifying outlier document behavior.
- **Leave-One-Out Ablation (Counterfactual Analysis)**
Measures document influence by observing output changes when removed.
- **SHA-256 Cryptographic Hashing**
Ensures document integrity and prevents corpus tampering.

Datasets

- **MS MARCO**
Large-scale dataset for training and evaluating retrieval systems.
- **Natural Questions (NQ-Open)**
Real-world query dataset for testing open-domain question answering.



Deployment / Infrastructure

- **Kaggle (T4 GPU, CUDA)**
Used for training and experimentation under constrained compute settings.
- **FAISS (CPU Memory Indexing)**
Enables fast retrieval without heavy GPU dependency.

Architecture Style

- **Secure RAG Pipeline (Zero-Trust Architecture)**
Treats all retrieved documents as untrusted and validates them before generation.
- **Multi-Signal Fusion Classifier**
Combines statistical, semantic, and behavioral signals for robust detection.



- [1] Tan, X., et al. (2024). *RevPRAG: Revealing Poisoning Attacks in Retrieval-Augmented Generation through LLM Activation Analysis*. arXiv:2411.18948.
URL: <https://arxiv.org/abs/2411.18948>
- [2] Zou, W., et al. (2024). *PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models*. arXiv:2402.07867.
URL: <https://arxiv.org/abs/2402.07867>
- [3] Zhang, B., et al. (2025). *Practical Poisoning Attacks against Retrieval-Augmented Generation (CorruptRAG)*. arXiv:2504.03957.
URL: <https://arxiv.org/abs/2504.03957>
- [4] Zhang, B., et al. (2025). *Benchmarking Poisoning Attacks against Retrieval-Augmented Generation*. arXiv:2505.18543.
URL: <https://arxiv.org/abs/2505.18543>
- [5] Xue, J., et al. (2024). *BadRAG: Identifying Vulnerabilities in Retrieval-Augmented Generation of Large Language Models*. arXiv:2406.00083.
URL: <https://arxiv.org/abs/2406.00083>
- [6] Clop, C., & Teglia, Y. (2024). *Backdoored Retrievers for Prompt Injection Attacks on Retrieval-Augmented Generation of Large Language Models*. arXiv:2410.14479.
URL: <https://arxiv.org/abs/2410.14479>
- [7] Cheng, Z., et al. (2025). *Secure Retrieval-Augmented Generation against Poisoning Attacks (RAGuard)*. arXiv:2510.25025.
URL: <https://arxiv.org/abs/2510.25025>
- [8] Su, J., et al. (2024). *Towards More Robust Retrieval-Augmented Generation: Evaluating RAG Under Adversarial Poisoning Attacks*. arXiv:2412.16708.
URL: <https://arxiv.org/abs/2412.16708>
- [9] Zhou, H., et al. (2025). *TrustRAG: Enhancing Robustness and Trustworthiness in Retrieval-Augmented Generation*. arXiv:2501.00879.
URL: <https://arxiv.org/abs/2501.00879>



- [10] Kim, S., et al. (2025). *Safeguarding RAG Pipelines with GMTP: A Gradient-based Masked Token Probability Method for Poisoned Document Detection*. arXiv:2507.18202.
URL: <https://arxiv.org/abs/2507.18202>
- [11] Zhang, B., et al. (2025). *Traceback of Poisoning Attacks to Retrieval-Augmented Generation (RAGForensics)*. arXiv:2504.21668.
URL: <https://arxiv.org/abs/2504.21668>
- [12] Edemacu, K., et al. (2025). *Filtration-Based Defense for Retrieval-Augmented Generation (FilterRAG)*. arXiv:2508.02835.
URL: <https://arxiv.org/abs/2508.02835>
- [13] Hong, G., et al. (2023). *Counterfactual Noise Defense for Open-Domain QA*. arXiv:2305.01579.
URL: <https://arxiv.org/abs/2305.01579>
- [14] Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401.
URL: <https://arxiv.org/abs/2005.11401>
- [15] *Fusion-based AI Security Study*. (2025). Information Fusion. DOI: 10.1016/j.inffus.2025.103900.
URL: <https://doi.org/10.1016/j.inffus.2025.103900>
- [16] Zhao, T., et al. (2024/2026). *Knowledge Poisoning Exploration in Information Fusion/RAG*. arXiv:2405.15556.
URL: <https://arxiv.org/abs/2405.15556>
- [17] Li, X., et al. (2024). *LLM Backdoor Detection via Chain-of-Scrutiny*. arXiv:2406.05948.
URL: <https://arxiv.org/abs/2406.05948>
- [18] Shafran, A., et al. (2024). *Denial-of-Service Jamming in RAG Databases*. arXiv:2406.05870.
URL: <https://arxiv.org/abs/2406.05870>



[19] Nazary, F., et al. (2025). *Recommender Data Poisoning (Poison-RAG)*. arXiv:2503.06254.

URL: <https://arxiv.org/abs/2503.06254>

[20] Shereen, E., et al. (2025). *Visual Document Poisoning in VD-RAG*. arXiv:2501.11759.

URL: <https://arxiv.org/abs/2501.11759>

[21] Xiang, et al. (2024). *Certifiably Robust Defense (RobustRAG)*. arXiv:2504.02132.

URL: <https://arxiv.org/abs/2504.02132>

[22] Liu, Y., et al. (2025). *Multimodal RAG Poisoning (Poisoned-MRAG)*. arXiv:2511.01268.

URL: <https://arxiv.org/abs/2511.01268>

[23] T. Zhao *et al.*, "Knowledge poisoning exploration in information fusion/RAG," *arXiv preprint arXiv:2405.15556*, 2024. [Online].

URL: <https://arxiv.org/abs/2405.15556>

[24] K. Edemacu *et al.*, "Defending against knowledge poisoning attacks during retrieval-augmented generation (FilterRAG)," *arXiv preprint arXiv:2508.02835*, 2025. [Online].

URL: <https://arxiv.org/abs/2508.02835>



THANK YOU